BATTLECTF QUALIFIER 2026 · OT · 142 PTS

RHEIN-VOGT STEELWORKS

Des identifiants par défaut au SSRF node-fetch, du denylist substring au balayage du réseau interne 172.25.0.0/24, de l'état du creuset indexé par session à la rupture : radiographie complète d'une chaîne d'exploitation OT en quatre actes.

Challenge OT#003 · Web / OT

Flag révélé par l'état du four après rupture du creuset, session unique

CTF{49b7d0bdb03d054b68dc793abae27fcf}

WRITEUP TECHNIQUE · OCTOBRE 2026

Sommaire

1. Vue d'ensemble du challenge 1

2. Reconnaissance et identifiants de mise en service 2

3. Étape 1 - SSRF : denylist substring et bypass hexadécimal 3

3.1 Un filtre substring, pas un parseur . . . 4

3.2 Rate-limit par session et rotation . . . 4

4. Étape 2 - Cartographie du réseau interne 172.25.0.0/24 4

5. Le piège - un état de four indexé par session 5

5.1 La vraie explication : deux compteurs, un même identifiant . . . 5

5.2 Répliques load-balancées : l'hypothèse écartée . . . 6

6. Étape 3 - La quête du /emergency_stop 6

7. Étape 4 - Rupture du creuset et lecture du flag 7

8. Leçons, impasses et artefacts 8

8.1 Quatre leçons transposables . . . 8

1. Vue d'ensemble du challenge

Rhein-Vogt Steelworks est un challenge OT (Operational Technology) de la battleCTF Qualifier 2026, coté 142 points. La cible est le portail d'exploitation d'une aciérie fictive, publié sur le port 6002 d'un hôte unique. Derrière cette façade opère une automatisation industrielle miniature : une API de four (furnace) développée en Express/Node sur le port 4000, un conteneur portal et un proxy openresty, tous isolés sur un réseau Docker interne 172.25.0.0/24, invisibles depuis Internet. L'objectif affiché tient en une phrase : provoquer la rupture du creuset (vessel) et lire le flag que cette catastrophe révèle dans l'état du système.

Champ	Valeur
Challenge	Rhein-Vogt Steelworks (OT#003) - battleCTF Qualifier 2026
Catégorie / points	OT / Web - 142 pts
Cible	http://169.58.126.107:6002
Stack	portail Node/Express derrière openresty - furnace API (Express, :4000) - portal (:3000) - openresty (:80) sur 172.25.0.0/24
Flag obtenu	CTF{49b7d0bdb03d054b68dc793abae27fcf}
Point d'entrée final	POST /emergency_stop sans ventilation préalable, via SSRF

Tableau 1 - Fiche d'identité du challenge

La chaîne gagnante se lit en quatre actes, chacun préparant le suivant, et aucune étape ne requiert de secret préalable : tout part d'un fichier de mise en service oublié. Les trois premières étapes ne coûtent qu'une poignée de requêtes, le rate-limiteur du portail étant contourné par une simple rotation de session. La dernière étape, elle, exige une discipline stricte : la totalité de l'enchaînement final doit s'exécuter dans une même session, sous peine de lire un état vierge.

- **1. Identifiants de mise en service.** /commissioning-notes.txt laisse traîner le compte operator / St33lW0rks!2024 et le ticket ORD-4471 : l'accès opérateur s'obtient sans attaque, par simple lecture d'un fichier statique.
- **2. SSRF node-fetch.** POST /ops/test-sensor (paramètres url, method, body) relaie une requête depuis le conteneur et renvoie {status, body} : un proxy HTTP interne aveugle mais bavard, armé par un compte opérateur standard.
- **3. Bypass hexadécimal.** Le denylist compare des sous-chaînes sur l'URL entière (furnace, localhost, 127.0.0.1, 172.17.0.1) ; http://0xac190002:4000 exprime 172.25.0.2 sous une forme numérique que la liste ne reconnaît pas.
- **4. Rupture puis lecture.** POST /emergency_stop sans ventilation préalable provoque vessel ruptured ; GET /status dans la même session révèle ruptured:true et le flag.

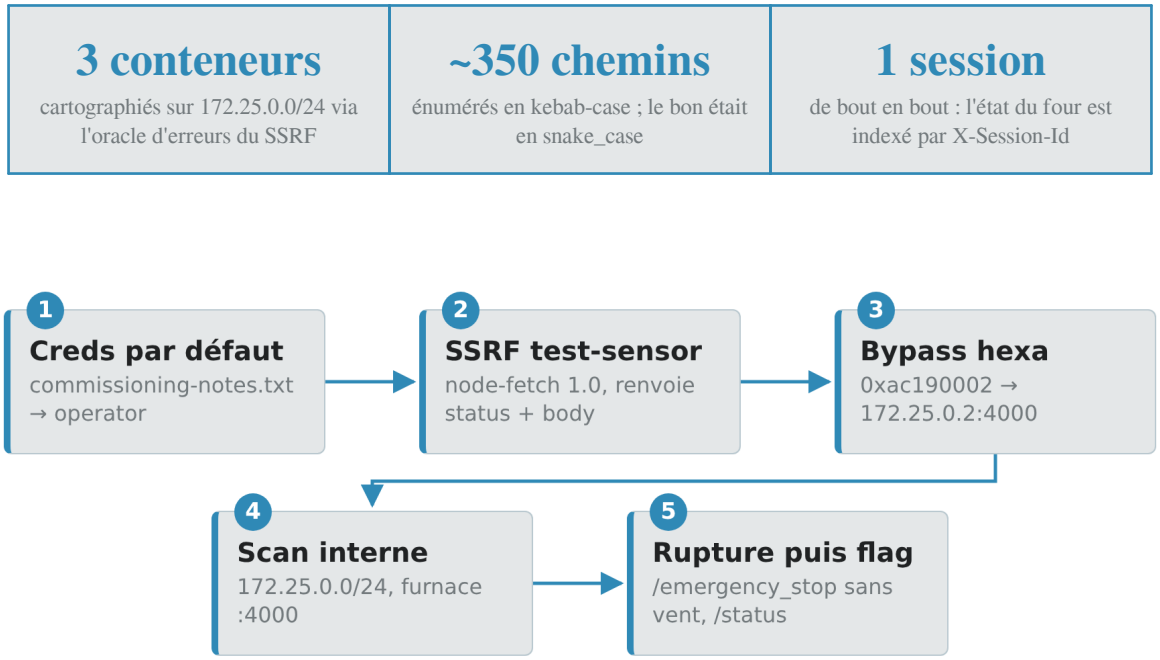


Figure 1 - Chaîne d'exploitation complète, du fichier de mise en service au flag

2. Reconnaissance et identifiants de mise en service

La surface externe est volontairement minuscule. La page d'accueil du portail ne propose qu'un formulaire de connexion, et un balayage des ports de l'hôte 169.58.126.107 révèle peu de choses : SSH (22), une plage 5000-5011 qui héberge d'autres challenges, la série 6000-6002 dont la nôtre, et quelques ports 702x. Rien qui ressemble à une API industrielle exposée. La première brèche n'est donc pas technique mais organisationnelle : un document de mise en service oublié à la racine du serveur web.

GET /commissioning-notes.txt renvoie un texte de commissioning qui mentionne le ticket ORD-4471 et, surtout, les identifiants provisoires du compte opérateur laissés le temps de la passation : operator / St33lW0rks!2024. Ce trope est la première leçon OT du challenge. Dans une vraie usine, les comptes provisoires créés pour la mise en service par le intégrateur sont un vecteur d'attaque documenté, et leur oubli après la réception des travaux est un classique des audits de sécurité industrielle : le compte reste actif, son mot de passe figure dans des documents papier ou numériques rarement nettoyés.

commissioning-notes.txt (extrait)

```
== Commissioning - Rhein-Vogt Steelworks ==
Ticket: ORD-4471
Portal account (temporary - rotate after handover):
username: operator
password: St33lW0rks!2024
```

Une fois connecté, le tableau de bord opérateur révèle l'unique outil d'administration du portail : POST /ops/test-sensor, un formulaire destiné à vérifier la joignabilité des capteurs du site. Sa présence dans l'interface d'un compte standard est déjà une anomalie - un outil de diagnostic réseau exécuté côté serveur est un point de départ de SSRF en puissance. Le formulaire n'accepte qu'un champ url ; un paramètre method et

un paramètre body s'ajoutent à la main, et le Content-Type de la requête relais est forcé à application/json. Aucun paramètre headers n'existe : impossible d'injecter des en-têtes arbitraires dans la requête sortante.

POST /ops/test-sensor - comportement observé

```
url = http://httpbin.org/get method = GET
-> 200 {"status": 200, "body": "{...réponse relayée...}"}
- node-fetch 1.0 côté serveur ; erreurs Node relayées telles quelles
(ECONNREFUSED, ETIMEDOUT, HPE_*...)
- la réponse interne inclut le statut ET le corps : un oracle complet
- Content-Type: application/json forcé ; pas de paramètre headers
```

Trois caractéristiques font de cet outil une arme complète plutôt qu'un gadget de diagnostic. Il relaie n'importe quelle URL sans restriction de schéma, il renvoie le statut et le corps de la réponse au lieu d'un simple acquittement, et il expose les erreurs Node brutes. Ces erreurs constituent un oracle d'information redoutable : ECONNREFUSED instantané signale un hôte vivant dont le port est fermé, ETIMEDOUT signale un hôte absent du réseau, et une erreur de parse HTTP (HPE_*) signale un service non HTTP qui accepte la connexion. Toute la cartographie du chapitre 4 reposera sur cet oracle, sans jamais disposer d'une exécution de code.

3. Étape 1 - SSRF : denylist substring et bypass hexadécimal

Le premier tir de sonde vers le service interne se solde par un refus sec : le portail filtre les URLs. La comparaison des réponses pour différentes cibles révèle la nature exacte du filtre : une denylist de sous-chaînes appliquée sur l'URL entière - pas sur l'hôte, pas sur une structure parsée, mais sur la chaîne de caractères brute envoyée dans le formulaire. Les termes furnace, localhost, 127.0.0.1 et 172.17.0.1 sont interdits, et le contrôle frappe même lorsqu'ils apparaissent dans le chemin de l'URL : une sonde vers http://httpbin.org/furnace est rejetée comme une sonde vers http://furnace:4000.

Sous-chaîne interdite	Exemple rejeté	Remarque
`furnace`	`http://furnace:4000/status`	le nom du conteneur cible, mais aussi tout path contenant le mot
`localhost`	`http://localhost:4000`	blocage du loopback par son nom
`127.0.0.1`	`http://127.0.0.1:4000`	blocage du loopback par son IP
`172.17.0.1`	`http://172.17.0.1:80`	la passerelle Docker par défaut

Tableau 2 - Denylist observée : des sous-chaînes, pas des hôtes

3.1 Un filtre substring, pas un parseur

Ce type de filtre est une illusion de sécurité. Il ne parse pas l'URL, ne normalise pas l'hôte, et ne raisonne pas en destination : il cherche quatre mots dans une chaîne. Toute représentation de l'adresse qui n'épelle pas littéralement ces mots passe la barrière. L'IPv4 n'est après tout qu'un entier de 32 bits : 172.25.0.2 vaut 0xAC190002 en hexadécimal, soit 2886799362 en décimal. node-fetch transmet ces formes exotiques à la couche réseau de Node, qui les normalise en 172.25.0.2 avant de connecter le socket - mais la denylist, aveugle aux bases numériques, laisse passer la chaîne http://0xac190002:4000/status sans reconnaître aucun terme interdit.

Contournement du denylist par forme hexadécimale

```
POST /ops/test-sensor
url = http://0xac190002:4000/status # 0xAC190002 = 172.25.0.2
-> 200 {"status": 200,
"body": "{\"temperature\":1550,\"pressure\":92,
\"vented\":false,\"ruptured\":false}"}
```

3.2 Rate-limit par session et rotation

L'outil de test est plafonné : enchaîner les sondes déclenche rapidement une réponse rate limited. Le compteur étant indexé par cookie de session, la parade est triviale - réauthentifier l'opérateur avant chaque requête crée une session neuve et un compteur vierge, et le coût d'une connexion de formulaires est négligeable. Ce détail, anodin en apparence, deviendra le piège central du challenge au chapitre 5 : car si le portail compte ses requêtes par session, la furnace API indexe aussi son état par session, et la rotation qui contourne le premier mécanisme sabote silencieusement le second.

4. Étape 2 - Cartographie du réseau interne 172.25.0.0/24

Une fois le denylist neutralisé, l'outil SSRF devient un scanner réseau complet. Chaque requête retourne en quelques centaines de millisecondes la différence entre un hôte vivant (ECONNREFUSED sur un port fermé) et un hôte fantôme (ETIMEDOUT après timeout). En balayant les passerelles Docker usuelles 172.17 à 172.31 puis les hôtes .2 à .5 de chaque sous-réseau vivant, la topologie émerge en quelques dizaines de requêtes : un seul réseau actif, 172.25.0.0/24, peuplé de trois conteneurs.

Hôte	Port ouvert	Identification
172.25.0.2	4000	furnace - API Express : 404 Cannot GET / typique, réponses JSON d'état
172.25.0.3	3000	portal - l'interface opérateur vue de l'intérieur
172.25.0.4	80	openresty - proxy pur, aucune route utile en énumération

Tableau 3 - Inventaire du réseau interne via l'oracle d'erreurs SSRF

Le scan des 36 ports candidats sur .2 et .4 - 80, 443, 8080, 3000-3002, 4000, 5000-5001, 9000, et les ports protocolaires OT emblématiques (502 Modbus, 4840 OPC UA, 20000 EtherNet/IP, 1883 MQTT, 5683 CoAP...) - ne révèle qu'un seul service exploitable : le port 4000 de 172.25.0.2. Ce port répond par le 404 Cannot GET / signature d'Express, accompagné de l'en-tête X-Powered-By qui confirme la stack. La cartographie des routes de la furnace API s'engage alors par sondes GET, POST et OPTIONS sur une liste de noms plausibles ; deux routes répondent immédiatement.

Furnace API (172.25.0.2:4000) - routes et état

```
GET http://0xac190002:4000/status
-> {"temperature":1550,"pressure":92,"vented":false,"ruptured":false}
POST http://0xac190002:4000/vent
-> {"ok":true,"pressure":35,"vented":true}
```

L'état du creuset tient en quatre champs : une température constante à 1550 degrés, une pression à 92, un drapeau vented et un drapeau ruptured. La logique attendue d'une aciérie est limpide : pour arrêter le four en sécurité, on ventile d'abord - le POST /vent fait chuter la pression à 35 et passe vented à true - puis on déclenche l'arrêt d'urgence. Le champ ruptured, jamais vrai par défaut, désigne sans ambiguïté l'objectif final : quelque part dans le code du serveur, sa mise à true doit réagir et, à n'en pas douter, dévoiler ce que le challenge cache.

5. Le piège - un état de four indexé par session

Pendant plusieurs heures, le challenge semble impossible. Après un POST /vent pourtant réussi - la réponse affiche bien pression 35 et vented:true - un GET /status immédiat renvoie l'état vierge : pression 92, vented:false. L'hypothèse d'une réinitialisation instantanée, le four qui se remet d'aplomb en moins d'une seconde, paraît confirmée par une étude minutée rigoureuse : vent à T0, puis sondes d'état à 0,6 s, 2 s, 5 s, 10 s, 30 s, 60 s, 120 s - toutes vierges, jusqu'à deux minutes après la ventilation.

L'étude minutée qui a trompé (chaque sonde sur session neuve)

```
T0.00 VENT : {"ok":true,"pressure":35,"vented":true}
T0.59 status : {"temperature":1550,"pressure":92,"vented":false,...}
T2.01 status : {...,"vented":false,...}
T5.14 status : {...,"vented":false,...}
T30.12 status : {...,"vented":false,...}
T120.09 status : {...,"vented":false,...}
```

5.1 La vraie explication : deux compteurs, un même identifiant

L'explication réelle est un couplage invisible entre deux mécanismes indexés par session. Le rate-limiteur du portail compte ses requêtes par cookie de session - d'où la rotation systématique mise en place pour contourner le plafond. Mais la furnace API indexe aussi son état par l'en-tête X-Session-Id, et l'outil /ops/test-sensor transmet fidèlement la session du portail au service interne. Chaque rotation de session lit donc un creuset différent, vierge, dans une table d'états séparée. Le vent ne disparaissait pas : il séjournait dans l'entrée de la table correspondant à sa session, tandis que chaque lecture interrogeait une entrée neuve.

Action	Session	État observé
POST /vent (pression 35)	session A	venté - mais personne ne le relit
GET /status (après rotation)	session B	vierge : entrée neuve de la table d'états
Conclusion erronée	sessions mélangées	« le vent se réinitialise en moins d'une seconde »
Chaîne correcte	session A de bout en bout	venté, puis ruptured, puis le flag

Tableau 4 - Mécanique de l'illusion : l'état suit la session, pas le serveur

5.2 Répliques load-balancées : l'hypothèse écartée

Avant de comprendre le couplage de sessions, l'hypothèse concurrente des répliques a été soigneusement testée : si plusieurs instances de la furnace API se partageaient la charge derrière un load-balancer, chaque lecture pourrait tomber sur un conteneur différent, expliquant des états incohérents sans reset magique. Quatorze GET /status parallèles tirés immédiatement après un vent n'ont montré aucun mélange d'états - soit toutes vierges, soit toutes ventées selon la session émettrice - et dix POST /vent simultanés ont produit des réponses identiques. Le load-balancing était écarté ; il restait à relier le X-Session-Id au tableau d'états, et la chaîne complète put enfin s'enchaîner dans une session unique.

6. Étape 3 - La quête du /emergency_stop

Restait à trouver le point de terminaison d'arrêt d'urgence. La furnace API ne documente rien : Cannot GET / ne liste aucune route, et les 404 d'Express ne distinguent pas une route existant en POST d'une route inexistante. Plusieurs centaines de noms candidats ont été sondés par OPTIONS, GET et POST : emergency-stop, emergencyStop, stop, shutdown, halt, abort, kill, panic, e-stop, estop, trip, melt-down, scaram - avec leurs déclinaisons de préfixes et de suffixes. Environ 350 chemins, tous en kebab-case ou en minuscules concaténées : les conventions « web moderne ». Le bon nom était /emergency_stop, en snake_case. Il n'a jamais été testé.

Ce n'est qu'en recoupant avec le writeup d'un autre joueur ayant résolu le challenge que le nom a été confirmé. L'ironie est cinglante : l'auteur a écrit un service Node/Express - un écosystème dont la convention dominante est le camelCase côté JavaScript, le kebab-case côté URLs - et a nommé sa route critique à la Python. La leçon dépasse l'anecdote : dans les systèmes OT réels, l'hétérogénéité de nommage est la norme, car chaque brique vient d'une époque, d'un langage et d'une équipe différents.

> *Un service Node/Express dont la route critique porte un nom à la Python : dans une aciérie, comme en CTF, le plan de dénomination des tags ne suit jamais une seule convention - il suit l'histoire des machines qui l'ont composé.*

Radical	Verbe	kebab-case	snake_case	camelCase	concaténé
emergency	stop	`emergency-stop`	`emergency_stop`	`emergencyStop`	`emergencystop`
e-stop	estop	`e-stop`	`e_stop`	`eStop`	`estop`
interlock	reset	`interlock-reset`	`interlock_reset`	`interlockReset`	`interlockreset`

Tableau 5 - Matrice de mutation : radical × verbe × convention de casse

L'énumération gagnante se décompose en trois réflexes que ce challenge illustre a contrario. Premièrement, croiser systématiquement chaque radical avec les quatre conventions de casse dominantes et les verbes d'action typiques du domaine : start, stop, open, close, reset, trip - seize variantes par paire radical-verbe, un coût mécanique dérisoire pour un balayage exhaustif. Deuxièmement, mobiliser les invariants du domaine OT : l'arrêt d'urgence - E-Stop - est un terme normalisé, imposé sur toute machine par la norme IEC 60204-1, et les écosystèmes PLC/SCADA, héritiers de Python et de C, nomment massivement leurs tags en snake_case. Troisièmement, préférer les wordlists éprouvées (seclists, raft, les listes d'endpoints API) à une liste faite main : elles couvrent d'emblée ces variantes de casse et de composition.

7. Étape 4 - Rupture du creuset et lecture du flag

La dernière étape tient en trois requêtes dans une session unique. La logique métier du four est implacable et constitue le twist final du challenge : ventiler PUIS arrêter d'urgence produit un arrêt propre - Furnace stopped safely - sans aucun flag. C'est l'arrêt d'urgence SANS ventilation qui provoque la rupture du creuset, et c'est cette catastrophe qui révèle le flag dans l'état du système. Le challenge punit littéralement le respect de la procédure de sécurité : la récompense est l'écart au process, précisément ce que l'attaquant d'un système industriel recherche.

Chaîne finale complète - une session operator unique, aucune rotation

```
# 1) état initial du creuset
GET http://0xac190002:4000/status
{"temperature":1550,"pressure":92,
"vented":false,"ruptured":false}
# 2) arrêt d'urgence SANS ventilation prealable
POST http://0xac190002:4000/emergency_stop
{"ok":false,"catastrophic":true,
"message":"vessel ruptured"}
# 3) relecture de l'état, même session : la rupture révèle le flag
GET http://0xac190002:4000/status
{"temperature":1550,"pressure":92,
"vented":false,"ruptured":true, ...}
-> CTF{49b7d0bdb03d054b68dc793abae27fcf}
```

Le variant propre confirme la logique : en refaisant la séquence avec un POST /vent préalable dans la même session, la réponse de /emergency_stop devient un ok:true accompagné de Furnace stopped safely et le creuset ne rompt pas. Le flag n'est donc pas la récompense de l'arrêt d'urgence, mais celle de la violation du mode opératoire - une ironie assumée de l'auteur, qui transforme une règle de sécurité fonctionnelle en

mécanique de jeu. C'est aussi une leçon OT sérieuse : les écarts au procédé (bypasser un interlock, sauter une étape de consigne) sont précisément les trajectoires qu'un attaquant cherche à provoquer, et les interdits métier font d'excellents marqueurs d'état pour un challenge.

Le flag CTF{49b7d0bdb03d054b68dc793abae27fcf} a été validé tel quel : il provient de la réponse live de GET /status après rupture, champ renvoyé au moment où ruptured passe à true. Aucune autre source n'existe sur l'instance - ni fichier statique, ni variable d'environnement exposée - la rupture du creuset est le seul canal de divulgation. Contrairement à beaucoup de challenges web, aucune course au rate-limit n'a été nécessaire en fin de chaîne : trois requêtes bien ordonnées dans une même session suffisent, la vraie difficulté ayant été de comprendre qu'elles devaient partager la même session.

8. Leçons, impasses et artefacts

Cette résolution a coûté plusieurs heures, dont l'essentiel dans deux impasses majuscules : l'illusion du reset instantané du vent, et l'énumération manquée du chemin snake_case. Les dresser avec leur verdict sert d'entraînement aux prochains challenges du même type - chaque piste écartée l'a été par un test, pas par intuition.

Piste explorée	Verdict
Injection d'en-têtes via test-sensor	écartée - pas de paramètre headers, Content-Type forcé
Répliques load-balancées de la furnace	écartée - 14 lectures parallèles homogènes après vent
Node inspector (9229, 9230, 5858) sur .2	écartée - ports fermés
Fuzz statique du portail (~250 noms × 6 extensions)	écartée - seul commissioning-notes.txt existe
Reset du vent en moins d'une seconde	illusion - état indexé par X-Session-Id, chaque sonde lisait une session neuve
Énumération de ~350 routes furnace en kebab-case	improductive - la route critique était en snake_case (/emergency_stop)

Tableau 6 - Impasses et pistes écartées, avec leur test de réfutation

8.1 Quatre leçons transposables

- **Denylist substring = bypass par représentation.** Un filtre qui cherche des mots dans une chaîne ne raisonne pas en destination : l'IPv4 étant un entier de 32 bits, les formes décimale et hexadécimale (0xac190002) contourment la liste tout en résolvant vers l'adresse cible.
- **Rate-limit par session = rotation.** Un compteur indexé par cookie se vide en réauthentifiant chaque requête ; identifier la clé d'indexation d'un mécanisme de limite est toujours la première étape de son contournement.

- **État interne indexé par un en-tête transmis = session unique de bout en bout.** Quand un outil relais transmet la session applicative au service interne, toute rotation lit un monde différent : la chaîne d'exploitation doit alors s'exécuter dans une même session, quitte à gérer le rate-limit autrement.
- **Énumération : croiser radicaux × verbes × conventions de casse.** Le domaine OT impose ses invariants - E-Stop normalisé par IEC 60204-1, tags PLC/SCADA en snake_case - et les wordlists éprouvées (seclists, raft) couvrent ces variantes mieux qu'une liste faite main.

Artefact	Rôle dans la session
`ot3_phase2.py ... ot3_phase21.py`	cartographie réseau, scans de ports, étude minutée, test de répliques
`ot3_net_phase2.json / ot3_subnets.json`	topologie découverte (172.25.0.0/24, 3 conteneurs)
`ot3_furnace_routes.json / ot3_furnace_paths.json`	énumération des routes de la furnace API
`OT3_FLAG.txt`	flag capturé : CTF{49b7d0bdb03d054b68dc793abae27fcf}
`cover.html / diagram.html / generate_body.py`	sources du présent writeup

Tableau 7 - Inventaire des artefacts produits pendant la session

Pour rejouer la résolution sur une instance fraîche du challenge, l'ordre est le suivant : récupérer les identifiants dans /commissioning-notes.txt, ouvrir une session operator et ne plus s'en déconnecter, puis dérouler les trois requêtes SSRF - état du creuset, arrêt d'urgence sans ventilation, relecture de l'état - en veillant à ce qu'aucune rotation de session ne s'intercale. Le rate-limit tolère confortablement trois requêtes ; la seule précaution réelle est la discipline de session. Sur une instance disputée, la rotation reste l'arme d'exploration des chapitres 3 et 4, mais elle doit impérativement s'arrêter à la porte de la chaîne finale.